SAHAYAK Continuous Sign Collection, Training & Deployment Guide

Copy-and-paste workflow for collecting full movement sequences, validating temporal model candidates, promoting safely, and pushing only the approved sequence model to GitHub.

Scope	Safe-by-default sequence
Windows local training project plus GitHub deployment repository.	Backup → collect → train candidate → test → promote → sync → Git commit/push → verify the deployed API.

1. Open the correct project

Run these commands in PowerShell from the local training project. The continuous system has a separate SQLite database and a separate model; this guide intentionally does not modify the static model or the legacy gesture_db.db.

```
Set-Location "C:\Users\user\3D Objects\New folder\SAHAYAK"
.\.venv\Scripts\Activate.ps1
python --version
Test-Path .\collect_sequences.py
```

If activation is blocked, keep the existing environment unchanged and use .\.venv\Scripts\python.exe instead of python for each command.

2. Back up the continuous database and active sequence model

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path $PWD "backups\continuous_$stamp"
New-Item -ItemType Directory -Path $backup -Force | Out-Null
foreach ($file in @(
  "continuous_gesture_db.db",
  "model\sahayak_sequence.joblib"
)) {
  if (Test-Path $file) { Copy-Item $file $backup -Force }
}
Write-Host "Backup saved to: $backup"
```

Close any collection process before copying the SQLite database. This backup protects the existing sequence samples and active continuous model. Do not copy over, rename, or delete gesture_db.db; it is a separate legacy database.

3. Collect full continuous-sign examples

```
python collect_sequences.py
```

- At the prompt, enter a stable class label, for example WHAT_IS_YOUR_NAME or THANK_YOU. Labels are normalized to uppercase with underscores.
- Press C once to START recording one full movement, perform the whole sign, then press C again to STOP and save. Repeat this C-start/C-stop cycle for many examples of the same label; then rerun the script for another label.
- Keep the start, middle, and end of the movement in frame. Use varied, natural examples and lighting. The collector saves valid variable-length sequences; it does not require a fixed 60-frame recording.
- The code requires at least two samples for each of at least two labels to train. The training script recommends 20-50+ varied samples per class as a better starting point; the holdout estimate is printed only when every class has at least five samples.

Check the actual local database counts (do not guess them):

```
python -c "from sequence_db import summary; print(summary())"
```

4. Train a candidate sequence model - active model stays unchanged

```
python train_sequence_model.py --db
```

The script loads continuous_gesture_db.db, validates samples, reports per-label counts, and prints a held-out result when the sample counts allow it. It writes model/sahayak_sequence_candidate.joblib; it does not replace model/sahayak_sequence.joblib unless --promote is explicitly supplied.

Run tests and validate the candidate shape/labels:

```
python -m unittest test_sequence.py test_sahayak.py
@'
import joblib
from pathlib import Path
new = joblib.load('model/sahayak_sequence_candidate.joblib')
p = Path('model/sahayak_sequence.joblib')
old = joblib.load(p) if p.exists() else None
labels = set(map(str, new.classes_))
missing = (set(map(str, old.classes_)) - labels) if old is not None else set()
assert getattr(new, 'n_features_in_', None) == 1320, 'wrong sequence feature count'
assert len(labels) >= 2, 'need two or more labels'
assert not missing, f'previous labels missing: {sorted(missing)}'
print('CONTINUOUS CANDIDATE OK:', sorted(labels))
'@ | python -
```

The model must accept 1,320 temporal features. Preserve all existing sequence labels when adding a new class. If any old label is missing, stop and inspect the database before promotion.

5. Promote only after tests and review

```
python train_sequence_model.py --db --promote
```

This command retrains and evaluates, backs up the current active sequence model into model/backups/, then replaces model/sahayak_sequence.joblib. It does not modify the static model, static CSV, or either database. Read its output and ensure promotion completes successfully.

Start the local API in one terminal:

```
python web_app.py
```

In another PowerShell window, inspect readiness and sample counts:

```
Invoke-RestMethod "http://127.0.0.1:5000/api/status" | ConvertTo-Json -Depth 8
Invoke-RestMethod "http://127.0.0.1:5000/api/sequence/samples" | ConvertTo-Json -Depth 6
```

- Confirm sequence_model_ready is True and sequence_sign_labels contains the new label plus all prior labels.
- Use the local website/camera or the deployed studio only after the promoted model responds correctly. Test the complete continuous movement, then test prior sequence classes and several static signs.
- If the candidate passes offline checks but live recognition fails, stop before GitHub deployment. Inspect the API response and console/Render logs; do not promote repeatedly without a clear reason.

6. Sync the promoted sequence model into the GitHub deployment tree

The training project and deployment clone must be separate folders for the sync helper. The helper copies only the active continuous model when ModelType is Continuous.

```
$TrainingRoot = "C:\Users\user\3D Objects\New folder\SAHAYAK"
$RepoPath = "C:\Users\user\3D Objects\New folder\SAHAYAK-ISL"

# First time only: clone the repository if it is not already cloned.
if (-not (Test-Path (Join-Path $RepoPath ".git"))) {
  git clone https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL.git $RepoPath
}

git -C $RepoPath status --short
# Continue only after you have safely dealt with any local changes.
git -C $RepoPath pull --ff-only origin main

# Save a remote backup branch before changing the model.
$backupBranch = "backup/pre-continuous-" + (Get-Date -Format "yyyyMMdd-HHmmss")
git -C $RepoPath branch $backupBranch main
git -C $RepoPath push origin $backupBranch

Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Continuous
```

The helper copies model/sahayak_sequence.joblib to SAHAYAK-Vercel-Deployment/backend/model/. It does not copy the static model, static label metadata, dataset, candidate model, SQLite database, or local backups.

7. Review, commit, and push with Git

```
git -C $RepoPath status --short
git -C $RepoPath diff --stat

# Stage ONLY the promoted continuous model:
git -C $RepoPath add `
  SAHAYAK-Vercel-Deployment/backend/model/sahayak_sequence.joblib

git -C $RepoPath diff --cached --name-only
# Confirm the staged list contains only the sequence model above.
git -C $RepoPath diff --cached --stat
git -C $RepoPath commit -m "Update continuous sign recognition model"
git -C $RepoPath push origin main
```

Do not stage continuous_gesture_db.db, data/landmarks_2hand.csv, gesture_db.db, model/backups, .venv, or candidate files. If unexpected paths are staged, stop and reset the staging area before adding only the intended model path.

8. Verify the deployed backend and test both modes

```
# Run after the Render deployment has finished.
$status = Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status"
$status.sequence_model_ready
$status.sequence_sign_labels
```

- Confirm sequence_model_ready is True and sequence_sign_labels includes the newly added sequence class and all earlier classes.
- Open https://sahayak-eight-lac.vercel.app/studio.html. Perform the complete continuous sign and test other sequence classes. Also check that the static-sign model still recognizes existing static signs.
- If the API status does not update, verify the pushed commit and wait for the Render deployment to complete; inspect Render logs before making a second model change.

Rollback rule

If the new model causes regressions, restore the prior sahayak_sequence.joblib from model/backups/, run the same sync helper, stage only the sequence model, and push a clearly labelled rollback commit. Keep the remote backup branch; never force-push main.